Technical Assignment

Build a Real-Time Stock Screener

Full Stack Engineer Evaluation

4–5 hours · Next.js 16 · TypeScript · Node.js

Domain context

A stock screener is a tool used by financial analysts and retail investors to filter a large universe of stocks down to a shortlist matching specific criteria. Instead of reviewing hundreds of companies manually, an analyst sets thresholds — for example: stocks with a P/E ratio below 20, market cap above $1B, and a positive price change today — and the screener returns only those that qualify.

Unlike a general dashboard, a screener is action-oriented. The analyst is looking for signals. Data freshness matters: a price that is 10 minutes stale can be meaningless in this context. The tool needs to feel live.

Financial terms you will encounter

- **Current price (c)** The latest traded price of a stock.
- **Previous close (pc)** Closing price from the last trading session. Used to calculate daily % change.
- **% change** Calculated as ((c − pc) / pc) × 100. Positive = stock is up today.
- **Market capitalisation** Total market value of all shares. Indicates company size (small-cap, mid-cap, large-cap).
- **P/E ratio** Price divided by earnings per share. A common valuation metric — lower often signals cheaper relative to earnings.
- **52-week high / low** Highest and lowest price over the past year. Shows where the stock sits in its historical range.
- **Volume** Number of shares traded today. High volume often confirms a price move is significant.

API you will use

Finnhub free-tier API — finnhub.io

Provides: stock quotes, company profiles, and a WebSocket feed for real-time price updates.

Sign up for a free API key at finnhub.io. The free tier has rate limits — design your

data-fetching strategy around this. Hitting limits is not an excuse; planning around them is part of the task.

The brief

A fintech startup wants a web-based stock screener for their in-house analysts. The tool should let an analyst load a list of stocks, see live prices updating in real time, apply filters to narrow the list, and get a quick AI-generated insight on any stock they are interested in — all without refreshing the page.

You are given the API, the technology constraints, and the broad requirements below. Everything else — what the UI looks like, which financial metrics to expose as filters, how you structure the data layer — is your decision. Document those decisions.

Important

The brief is intentionally lean on specifics. Part of this task is figuring out what a useful stock screener needs, what data matters, and what is realistic to build well in 4–5 hours.

A well-reasoned trade-off you did not build is worth more than a half-working feature you did.

What needs to be built

1 Project setup and rendering strategy

Architecture decision — do this first, before writing any components

- Scaffold a Next.js 16 app using the App Router with TypeScript. Use npx create-next-app@latest — this will scaffold a Next.js 16 project by default with TypeScript, Tailwind, and ESLint preconfigured.
- Turbopack is the default bundler in Next.js 16 for both dev and production builds. You do not need any flags. If you have a reason to fall back to webpack, document it.
- Next.js 16 makes caching entirely opt-in. All dynamic code runs at request time by default. To cache a page, component, or function, use the new 'use cache' directive explicitly. There is no implicit caching. Plan your caching strategy around this and document it in DECISIONS.md.
- middleware.ts is deprecated in Next.js 16. Rate limiting and request interception should be implemented in proxy.ts, which exports a proxy() function and runs on Node.js. Document why this boundary matters.
- React Compiler is now stable in Next.js 16 and handles memoization automatically. Enable it in next.config.ts. You should not need manual React.memo or useMemo in most cases — if you add them, justify why in a code comment.
- Decide the correct rendering strategy for each route — static, dynamic, ISR, or PPR — and document the reasoning in DECISIONS.md. Different routes will warrant different strategies.
- Add Suspense boundaries with skeleton loading UI for any route that fetches data.

What this tests: Does the candidate know Next.js 16 changed caching, middleware, and memoization defaults — or do they write Next.js 14 patterns and wonder why nothing works as expected?

2 Live stock list with real-time price updates

Core feature — this is the heart of the screener

- Display a list of at least 20 stocks showing: ticker symbol, company name, current price, % change today, and at least one additional metric of your choice.
- Connect to the Finnhub WebSocket feed (wss://ws.finnhub.io) to receive real-time price updates. Prices must update on screen without any page reload or polling.

- Handle the WebSocket lifecycle correctly — connection, reconnection on drop, and cleanup on unmount.
- The UI must remain smooth and responsive even as price updates arrive rapidly. Think about how frequently you commit updates to the DOM.

What this tests: Real-time state management under continuous data flow. Naive useEffect + setState on every tick will cause visible jank.

3 Screener filters — URL-driven

State management and shareability

- Add at least 3 filters that an analyst would actually use. Choose metrics that make financial sense — not just arbitrary number ranges. Justify your choices in DECISIONS.md.
- All active filters must be reflected in the URL as query parameters. A filtered view must be fully reproducible by sharing the URL.
- Filters must survive a hard page refresh. Use useSearchParams or equivalent.
- Debounce any text/range inputs. Do not trigger filtering on every keystroke.

What this tests: URL-as-state is a common gap. Most engineers reach for useState by reflex. This tests whether that reflex exists.

4 Stock detail view

Information architecture — you decide what matters

- Clicking any stock opens a detail view (page or panel — your call).
- Show expanded information about the stock. You decide what 'more information' means for an analyst — think about what someone making a buy/sell decision might want to see.
- Choose the right rendering strategy for this view independently of the main listing. Document why.
- The detail view must have a shareable URL.

What this tests: Domain reasoning. Candidates who show genuine thought about what financial information is useful will stand out.

5 AI-powered stock insight

AI integration with streaming

- Each stock should have a way to generate a short AI-written insight — a 2–3 sentence analyst-style commentary on what the data suggests.
- Build a Next.js API route (/api/insight) that accepts a stock's data and calls an LLM of your choice (OpenAI, Anthropic, or any free-tier model).
- The insight must stream token-by-token into the UI using ReadableStream. A full response appearing after a delay is not acceptable.
- Cache generated insights so the same stock is not re-called on every click. Cache invalidation strategy is your call — document it.
- If the AI call fails, the rest of the screener must continue working normally. Failures must be handled gracefully.

What this tests: Streaming from a Next.js route is non-trivial. Error isolation is even more so. Candidates who let AI failures crash the main UI are showing a production blind spot.

6 API layer and data design

Backend fundamentals

- Create a typed Next.js API route that proxies the Finnhub API — normalise field names, add computed fields (e.g. percentChange, priceVs52wHigh), and strip unused fields before sending to the client.
- Add input validation using Zod (or equivalent) on all query parameters.
- Implement a basic rate limiter in proxy.ts — Next.js 16 replaces middleware.ts with proxy.ts for request interception. Use this to enforce a limit of 30 requests per minute per IP using an in-memory map. Note any production limitations of in-memory rate limiting in DECISIONS.md.
- Document all API routes in API.md: endpoint, accepted params, response shape, and any caching behaviour.

What this tests: API hygiene and awareness of Next.js 16's proxy.ts boundary. Candidates who still use middleware.ts or skip validation entirely are showing a gap.

Hard constraints

These are non-negotiable

Next.js 16 with App Router only — use npx create-next-app@latest to scaffold. No Pages Router.

TypeScript throughout — no ‘any’ without an inline comment explaining why.

Caching must use the 'use cache' directive explicitly — there is no implicit caching in Next.js 16. Every cached route, component, or function must be opted in deliberately.

Request interception must use proxy.ts — middleware.ts is deprecated in Next.js 16. Your rate limiter must live in proxy.ts.

React Compiler must be enabled — do not add manual React.memo or useMemo without a justified inline comment. If you add them, explain why the compiler is insufficient for that case.

Real data only — no mocked or hardcoded prices. The Finnhub WebSocket feed must be live.

No UI component libraries — Tailwind is allowed, but no shadcn, MUI, Chakra, Radix UI, or similar. Build your own components.

Deliverables

Submit all of the following. Incomplete submissions will be evaluated on what is present, but missing items will be noted.

- **Public GitHub repo** with a meaningful commit history. A single large commit suggests the work was not done iteratively.
- **README.md** setup instructions, which API keys are needed, and any known issues or limitations.
- **DECISIONS.md** rendering strategy rationale for each part of the app, how you handled real-time state, which filters you chose and why, what you cut in the interest of time, and any trade-offs you would revisit.

- **API.md** each endpoint, accepted parameters, response shape, and caching behaviour.
- **Bundle analyser screenshot** run next build with @next/bundle-analyzer and include the output. Comment on anything notable.
- **Loom or screen recording (2–3 min)** show the screener working live with prices updating, filters applied, the detail view, and the AI insight streaming.

Evaluation rubric

Each area below will be assessed at one of three levels. We are not looking for perfection — we are looking for evidence of production-level thinking.

Area	Needs work	Good	Strong
Rendering strategy	SSR used everywhere, no 'use cache' directive used	Correct mode chosen, some caching in place	Correct + 'use cache' used explicitly + written rationale for each route
Real-time architecture	Polling or setTimeout used instead of WebSocket	WebSocket connected, prices update on screen	WebSocket + clean state management, UI stays responsive under load
Domain understanding	Arbitrary or irrelevant filter metrics	Reasonable metrics (price change, volume)	Metrics make analytical sense — P/E, market cap, 52W range — with a brief explanation of why
AI integration	Basic fetch call, no streaming	Streaming works, token-by-token in UI	Streaming + in-memory cache + AI failure never breaks main listing
API design	Pass-through proxy, no types or validation	Typed routes, Zod validation on params	Typed + validated + rate limited + documented in API.md
Code quality & thinking	Works but messy, one large commit	Clean TypeScript, logical commit history	Clean code + commits tell a story + DECISIONS.md reads like a production engineer

What a strong submission looks like

Green flags

Uses 'use cache' on company profiles (they don’t change every second) but keeps the WebSocket price feed fully dynamic — and explains the split in DECISIONS.md.

Implements rate limiting in proxy.ts, not middleware.ts — shows awareness of the Next.js 16 network boundary change.

Enables React Compiler and avoids scattering manual React.memo — or adds them with a clear comment explaining why the compiler isn’t sufficient.

Filter choices reflect financial reasoning: P/E, market cap, % change — not just ‘min price / max price’.

URL state is used for filters without being asked. The candidate knows shareability is a requirement.

The AI insight fails gracefully — an error badge replaces the button, the rest of the app keeps running.

DECISIONS.md sounds like an engineer who has been in production, not someone following a tutorial.

Notes that in-memory rate limiting in proxy.ts does not survive horizontal scaling — flags it proactively without being asked.

Red flags

Everything is a Client Component with useEffect for data fetching — shows no understanding of the server-first model.

Uses implicit fetch caching or next.revalidate without the 'use cache' directive — that model no longer exists in Next.js 16.

Rate limiting placed in middleware.ts instead of proxy.ts — deprecated in Next.js 16.

Manual React.memo and useMemo everywhere with no justification — React Compiler makes most of this unnecessary.

WebSocket implemented with polling as a fallback without acknowledgment of the trade-off.

Filters live in useState and are lost on refresh.

AI call failure causes the whole listing to break or show an error state.

DECISIONS.md is a list of technologies used, not decisions made.

Single commit with all the code.

Final note

We will ask you to walk through your DECISIONS.md in the follow-up interview. Come prepared to justify every choice — including what you cut and why. A well-reasoned trade-off you did not build is worth more than a feature you built without knowing why.

Good luck.